

8218-Arsh Nadeem-Introduction to java 2-Jan 2025

1. Write a program to display values of enums using a constructor & getPrice() method (Example display house & their prices)

-> here we have to make an enum class 'House' and a main class 'EnumExample'

```
Main.java  House.java  EnumExample.java
1  public enum House { 2 usages
2
3      APARTMENT( price: 50000), no usages
4      FLAT( price: 150000), no usages
5      BUNGALOW( price: 250000), no usages
6      FARMHOUSE( price: 120000); no usages
7
8  // to store prices, making final so prices cannot be altered
9      private final int price; 2 usages
10
11     // constructor for price initialization
12     House(int price) { 8 usages
13         this.price = price;
14     }
15
16     // Getting the price
17     public int getPrice() { 1 usage
18         return price;
19     }
20
21 }
```

```
Main.java  House.java  EnumExample.java
1  public class EnumExample {
2      public static void main(String[] args) {
3          System.out.println("House with prices: ");
4
5          //finding all prices
6          for (House h : House.values()) {
7              System.out.println(h + " costs in rupee " + h.getPrice());
8          }
9      }
10 }
11
```

Output:

```
/usr/lib/jvm/java-21-openjdk-amd64/bin
House with prices:
APARTMENT costs in rupee 50000
FLAT costs in rupee 150000
BUNGALOW costs in rupee 250000
FARMHOUSE costs in rupee 120000

Process finished with exit code 0
```

2. Create a User class with fields: firstname, lastname, age, phone number. Write a program which accepts values of user fields from commandline, create object and append that to a text file. After every user creation the program should prompt: "Do you want to continue creating users? (Type QUIT to exit)" and keep on accepting values and writing to file until user quits.

-> creating Userdetail class and and User class

```
UserDetails.java ×
1  import java.io.BufferedWriter;
2  import java.io.FileWriter;
3  import java.io.IOException;
4  import java.util.Scanner;
5
6  class User{ 1 usage
7      private String fname, lName, phone; 2 usages
8      private int age; 2 usages
9
10     public User(String fname, String lName, String phone, int age) { 1 usage
11         this.fname = fname;
12         this.lName = lName;
13         this.phone = phone;
14         this.age = age;
15     }
16
17     @Override 9 usages
18     public String toString() {
19         return "User{" +
20             "fname=" + fname + '\n' +
21             ", lName=" + lName + '\n' +
22             ", phone=" + phone + '\n' +
23             ", age=" + age +
24             '}';
25     }
26 }
```

```

28
29 public class UserDetails {
30     public static void main(String[] args) {
31
32         Scanner sc = new Scanner(System.in);
33         // I will use try with resources which will automatically close the resource.
34         try(BufferedWriter writer = new BufferedWriter(new FileWriter( fileName: "user.txt", append: true))){
35             // loop will execute until stopped
36             while(true) {
37                 System.out.print("First name: ");
38                 String fname = sc.next();
39                 System.out.print("Last name: ");
40                 String lname = sc.next();
41                 System.out.print("Age: ");
42                 int age = sc.nextInt();
43                 System.out.print("Phone number: ");
44                 String phone = sc.next();
45                 // After reading age and phone, I added an extra sc.nextLine()
46                 // to consume the leftover newline.
47                 // This clears the buffer and ensure the next sc.nextLine()
48                 // actually waits for user input. otherwise it will skip
49                 sc.nextLine();

```

-> writing in file

```

51                 writer.write(new User(fname, lname, phone, age).toString());
52                 writer.newLine();
53
54                 System.out.println("adding user successfully");
55                 System.out.println("type QUIT to exit");
56                 String choice = sc.nextLine();
57                 if (choice.equals("QUIT")) break;
58             }
59         }catch (IOException | NumberFormatException e){
60             System.out.println("error" + e.getMessage());
61         }finally{
62             sc.close();
63         }
64         System.out.println("Exiting...");
65     }
66 }

```

Output:

```
/usr/lib/jvm/java-21-openjdk-amd64/bin/java -javaagent:/opt/intellij/lib/idea_rt.jar=389
First name: Abhinav
Last name: Srivastav
Age: 32
Phone number: 80380183194
adding user successfully
type QUIT to exit

First name: Arsh
Last name: Nadeem
Age: 23
Phone number: 31478937443
adding user successfully
type QUIT to exit
QUIT
Exiting....
```

-> checking the file content. This file will be permanently saved.

```
arsh@TTNPL-arshnadeem:introduction to java2 (master)$ ls -l
total 16
-rw-rw-r-- 1 arsh arsh 423 Feb  4 21:43 'introduction to java2.iml'
drwxrwxr-x 3 arsh arsh 4096 Feb  4 21:50 out
drwxrwxr-x 2 arsh arsh 4096 Feb  4 22:36 src
-rw-rw-r-- 1 arsh arsh 195 Feb  4 22:38 user.txt
arsh@TTNPL-arshnadeem:introduction to java2 (master)$ cat user.txt
User{fname='Arsh
, lname='Nadeem
, phone='38208241
, age=23}
User{fname='Abhinav
, lname='Srivastav
, phone='80380183194
, age=32}
User{fname='Arsh
, lname='Nadeem
, phone='31478937443
, age=23}
arsh@TTNPL-arshnadeem:introduction to java2 (master)$
```

3. Write a program to count the number of occurrences of a word in a file. The file name and word should be supplied through commandline.

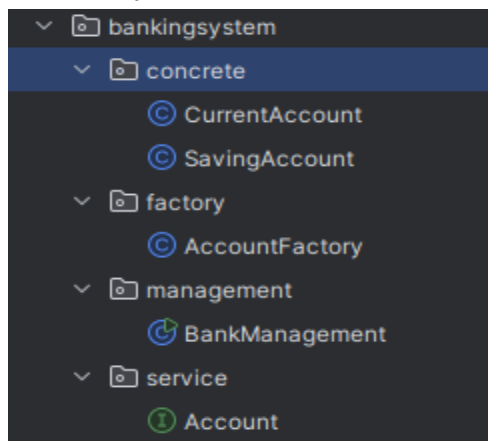
```
1 import java.io.BufferedReader;
2 import java.io.FileReader;
3 import java.io.IOException;
4
5 public class Count {
6     @ public static void main(String[] args) {
7         // if argument length is less than 2 means we will not get our desired output
8         if(args.length < 2){
9             System.out.println("not enough arguments");
10            return;
11        }
12        String fileName = args[0];
13        String word = args[1];
14        int count = 0;
15
16        // using try with resources to auto close
17        try(BufferedReader reader = new BufferedReader(new FileReader(fileName))){
18            String line;
19            // checking that we reached end of file or not,
20            // if not split and add words to arrays of string
21            while((line = reader.readLine()) != null){
22                // splitting white spaces
23                String[] words = line.split("regex: "\\s+");
24                for (String w : words){
25                    if(w.equals(word)){
26                        count++;
27                    }
28                }
29            }catch (IOException e){
30                System.out.println("error in reading this file " + e.getMessage());
31            }
32
33            System.out.println("Word " +word+" appers "+count+" times");
34        }
35    }
```

Output:

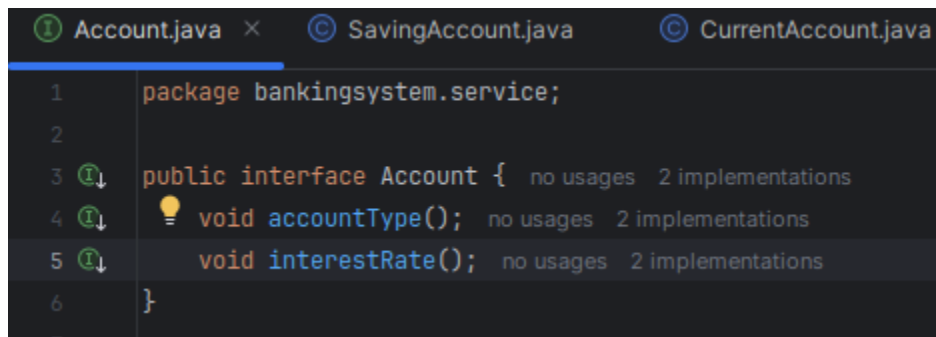
```
arsh@TTNPL-arshnadeem:src (master)$ cat example.txt
Hello Arsh, Arsh how are you
Hello Arsh, how are you. Hello is Abhinav Sir your Mentor
arsh@TTNPL-arshnadeem:src (master)$ javac Count.java
arsh@TTNPL-arshnadeem:src (master)$ java Count example.txt Hello
Word Hello appers 3 times
arsh@TTNPL-arshnadeem:src (master)$ java Count example.txt
not enough arguments
```

4. Write a program to show application of Factory Design Pattern.

->Directory structure:



->interface : I will define common methods for all accounts.



-> concrete includes the types of accounts i.e. concrete classes

* implementing all methods of interface 'Account'

->Saving account details



-> CurrentAccount details

```
Account.java    CurrentAccount.java    AccountFactory.java
1 package bankingsystem.concrete;
2
3 import bankingsystem.service.Account;
4
5 public class CurrentAccount implements Account { 2 usages
6     @Override 1 usage
7     public void accountType() {
8         System.out.println("For bussiness and traders");
9     }
10
11     @Override 1 usage
12     public void interestRate() {
13         System.out.println("No interest");
14     }
15 }
```

AccountFactory : It will return the correct object based on input.

```
AccountFactory.java    BankManagement.java
1 package bankingsystem.factory;
2
3 import bankingsystem.concrete.CurrentAccount;
4 import bankingsystem.concrete.SavingAccount;
5 import bankingsystem.service.Account;
6
7 public class AccountFactory { 2 usages
8     @ public static Account createAccount(String type){ 1 usage
9         if(type.equalsIgnoreCase( anotherString: "Saving")){
10             return new SavingAccount();
11         } else if (type.equalsIgnoreCase( anotherString: "Current")) {
12             return new CurrentAccount();
13         } else {
14             throw new IllegalArgumentException("Invalid account type");
15         }
16     }
17 }
```

- > management i.e main class
- > assigning type of account to account based on the input
- > displaying the information i.e. account and rate

```
AccountFactory.java  BankManagement.java x
4  import bankingsystem.service.Account;
5
6  import java.util.Scanner;
7
8  public class BankManagement {
9      public static void main(String[] args) {
10         Scanner sc = new Scanner(System.in);
11
12         System.out.println("<-----Welcome to our Bank----->");
13         System.out.print("Enter account type:");
14         String accountType = sc.nextLine();
15
16         Account account = AccountFactory.createAccount(accountType);
17         account.accountType();
18         account.interestRate();
19         sc.close();
20     }
21 }
```

Output:

```
/usr/lib/jvm/java-21-openjdk-amd64/bin/java -javaagent:/opt/intellij/lib/idea_rt.jar=36467:/opt
<-----Welcome to our Bank----->
Enter account type:saving
For general users
Interest rate 7%
<-----Welcome to our Bank----->
Enter account type:current
For bussiness and traders
No interest
<-----Welcome to our Bank----->
Enter account type:
```


5. Write a program to show application of Singleton Design Pattern.

```
public class SingletonExample {  
    public static void main(String[] args) {  
  
        DatabaseConn db1 = DatabaseConn.getInstance();  
        db1.message();  
  
        DatabaseConn db2 = DatabaseConn.getInstance();  
        db2.message();  
  
        System.out.println(db1 == db2);  
    }  
}
```

-> The static variable (instance) belongs to the class, not any specific object. By this, all objects share the same instance.

-> We call DatabaseConn.getInstance() without creating an object. This prevents multiple instances from being created.

-> Since only one instance exists, it avoids unnecessary memory usage

```
SingletonExample.java ×  
14  
15 class DatabaseConn{ 7 usages  
16 // private: Prevents direct modification from outside the class  
17 // static: Ensures a single shared instance across all objects  
18 private static DatabaseConn dbConn; 3 usages  
19 // this constructor is private so it cannot be accessed from outside the class  
20 private DatabaseConn(){ 1 usage  
21     System.out.println("Initializing db connection");  
22 }  
23 // making it static so that :The static variable (instance) belongs to the class,  
24 // not any specific object. This way, all objects share the same instance.  
25 public static DatabaseConn getInstance(){ 2 usages  
26     if(dbConn == null){  
27         dbConn = new DatabaseConn();  
28     }  
29     return dbConn;  
30 }  
31  
32 public void message(){ 2 usages  
33     System.out.println("Connected to db");  
34 }  
35 }
```

```
/usr/lib/jvm/java-21-openjdk-amd64/bin/java -javaagent:/opt/intellij/lib/idea_rt.jar=34055:./
Initializing db connection
Connected to db
Connected to db
true
```